

Authors

R4176c Computer Vision

Бахорина Ольга Александровна, ИСУ 506086

The task and guidelines were prepared by Andrei Zhdanov and Sergei Shavetov, ITMO University, 2025.

Practical Assignment No 1. Image Segmentation

Study of the basic methods for image segmentation into semantic areas.

To implement the current practical assignment task we would need OpenCV and NumPy libraries along with image display functions we wrote during the Image Processing class. We import OpenCV's `cv2` library as `cv` for easier use.

```
In [1]: # Import OpenCV library both as cv and cv2
import cv2
import cv2 as cv
# Import NumPy library as np
import numpy as np
# Import sys for floats epsilon value
import sys
# Import scikit-image for an entropy filter
import skimage.filters.rank
import skimage.morphology
# Import functions from our utility library
from pa_utils import ShowImages, exit
import warnings
warnings.simplefilter("ignore")
```

Introduction

In the current practical assignment we will learn some basic segmentation methods. These will be various image intensity thresholding methods and color segmentation methods.

Task 1. Binarization

Take an arbitrary image. Perform the image binarization using the considered methods. Depending on the image, use upper or lower threshold binarization.

The simplest way to segment an image into two classes (background and object pixels) is by *binarization*. In this case we will convert our image to black-and-white color model by applying threshold to a image pixel intensity values. There are several common approached how it can be done, so let's check them out.

The general image binarization algorithm would be as follows:

1. Load an image.
2. Convert it to grayscale.
3. Threshold the grayscale image.

1.1 Simple Thresholding

We will try the thresholding algorithms on an arbitrary color image. Let's read it in BGR and then convert to a GRAYSCALE mode.

```
In [2]: # Read an image from a file in BGR
fn = "images/lena_color.png"
I1 = cv.imread(fn, cv.IMREAD_COLOR)
if not isinstance(I1, np.ndarray) or I1.data == None:
    print("Error reading file \"{}\\".format(fn))
    exit()

# Convert loaded BGR image to grayscale
I1gray = cv.cvtColor(I1, cv.COLOR_BGR2GRAY)
# Display it
ShowImages([("Source image", I1),
             ("Grayscale", I1gray)], 2)
```

Source image



Grayscale



1.1.1 Single thresholding

The first and the most simple binarization method is the single thresholding method which is described with the following formula:

st

where

- is the source image;
- `st` is the single threshold binarized image;
- is the binarization threshold.

In OpenCV, the thresholding operation is executed with `cv2.threshold(image, t, maxval, method)` function. It takes an `image` to process, the threshold parameter `t`, the value to set for thresholded image pixels `maxval`, and the thresholding `method` as its parameters. It returns a tuple with the threshold used during thresholding and the thresholded image.

The following single thresholding methods are supported in OpenCV:

- `cv2.THRESH_BINARY` - set everything greater than `value` to the `maxval` and everything else to `0` :

binary

- `cv2.THRESH_BINARY_INV` - set everything greater than `value` to the `0` and everything else to `maxval` :

binary_inv

- `cv2.THRESH_TRUNC` - truncate everything greater than `value` to `value` while keeping other intact:

trunc

- `cv2.THRESH_TOZERO` - threshold everything less or equal than `value` to `0` while keeping other intact:

tozero

- `cv2.THRESH_TOZERO_INV` - threshold everything greater than `value` to `maxval` while keeping other intact:

tozero

Simple threshold binarization according to the formula above is executed by using the `cv2.THRESH_BINARY` method.

Let's try executing the single thresholding with a threshold value of `127` to segment every pixel above the threshold to `255` and every pixel below the threshold to `0`.

```
In [3]: # Single thresholding
t = 127
ret, I1st = cv.threshold(I1gray, t, 255, cv.THRESH_BINARY)
ShowImages(["Single thresholding", I1st])
```

Single thresholding



By varying the thresholding parameter `t` we can shift the binarization boundary.

1.1.2 Double Thresholding

In some cases, the single thresholding is not enough, for example, if we need to select a region. In this case, the double (range) thresholding is used. The double thresholding method is described with the following formula:

$$dt = \begin{cases} 1 & \text{if } I_1 \leq t_1 \text{ and } I_1 \leq t_2 \\ 0 & \text{otherwise} \end{cases}$$

where

- I_1 is the source image;
- dt is the double threshold binarized image;
- t_1 is the lower binarization threshold;

- t_2 is the upper binarization threshold.

OpenCV does not provide a special function for double thresholding, however, it can be done by the subsequent call of two threshold functions with different methods:

1. At first, should set all values above t_2 to zeros while keeping other intact (`THRESH_TOZERO_INV` method);
2. Secondly, should set all values below t_1 to zeros (`THRESH_BINARY` method).

Let's try executing the double thresholding with `range` to segment every pixel within this range to `and` every pixel outside of this range to `.`

```
In [4]: # Double thresholding
t1 = 127
t2 = 200
ret, I1dt = cv.threshold(I1gray, t2, 1, cv.THRESH_TOZERO_INV)
ret, I1dt = cv.threshold(I1dt, t1, 1, cv.THRESH_BINARY)
ShowImages(["Double thresholding", I1dt])
```

Double thresholding



By varying the thresholding parameters `t1` and `t2` we can shift the binarization range borders.

1.1.3 Self-work

Self-work

Take some arbitrary image and threshold it with **single and double** thresholding methods. Display the results.

```
In [5]: src = cv.imread("images/scene_gray.jpg", cv.IMREAD_GRAYSCALE)

low_t = 100
res_single = np.where(src > low_t, 1, 0).astype(np.uint8)

band = (60, 130)
res_double = np.zeros_like(src, dtype=np.uint8)
for y in range(src.shape[0]):
    for x in range(src.shape[1]):
        v = src[y, x]
        if band[0] < v <= band[1]:
            res_double[y, x] = 1

ShowImages([("Source (camera)", src),
             (f"Single t={low_t}", res_single),
             (f"Double {band}", res_double)], 3)
```

Source (camera)



Single t=100



Double (60, 130)



1.2 Automatic Thresholding

Binarization thresholds t_1 and t_2 can either be set manually or calculated using special algorithms. In the case of automatic threshold calculation, the following algorithms can be used.

1.2.1 Mean value

1. Find the maximum $\max$ and minimum $\min$ intensity values of the original grayscale image and find their arithmetic mean. The arithmetic mean will be the global binarization threshold :

$$\frac{\max + \min}{2}$$

1.2.2 Intensity gradient weighting

Find the optimal threshold based on the modulus of each pixel intensity gradient. For this, at first, it is required to calculate the modulus of the gradient at each image point :

then calculate the optimal threshold value :

$$\frac{\sum_{x=0}^{X-1} \sum_{y=0}^{Y-1} p_{xy}}{\sum_{x=0}^{X-1} \sum_{y=0}^{Y-1} 1}$$

1.2.3 Otsu method

The optimal threshold can be calculated by the statistical Otsu method [1] which splits all pixels into two classes and . This method minimizes the variance within each class σ_1^2 and σ_2^2 and maximizes the variance between classes.

The algorithm for calculating the threshold by the Otsu method consists of the following steps:

1. Compute an image histogram of intensities, and probabilities $p_i = \frac{n_i}{N}$ for each intensity level, where n_i is the number of pixels with intensity level i , and N is the number of pixels in the image.
2. Set the initial threshold $t = 0$ and threshold $t = L$, which divides all pixels into two classes, where L is the maximum value of the image intensity.
3. In the loop for each value of threshold from $t = 0$ to $t = L$:

A. Compute probabilities of two classes p_1 and p_2 , and arithmetic mean μ_1 and μ_2 , where $\mu = \sum_{i=0}^L i p_i$:

$$\begin{aligned} p_1 &= \sum_{s=0}^t p_s, \quad \mu_1 = \frac{\sum_{s=0}^t s p_s}{p_1} \\ p_2 &= \sum_{s=t+1}^L p_s, \quad \mu_2 = \frac{\sum_{s=t+1}^L s p_s}{p_2} \end{aligned}$$

2. Calculate the interclass variance σ_b^2 :

$$\sigma_b^2 = p_1 \mu_1^2 + p_2 \mu_2^2 - (\mu_1 p_1 + \mu_2 p_2)^2$$

3. If the calculated value σ_b^2 is greater than the current value σ_b^2 , then assign the value of the interclass variance to the threshold $t = \frac{\sigma_b^2}{\sigma_b^2 + 1}$.

4. The optimal threshold corresponds to the maximum $\frac{2}{b}$.

References

1. Otsu, N. (1979). A threshold selection method from gray-level histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9 (1), 62–66.
<https://doi.org/10.1109/TSMC.1979.4310076>

In OpenCV thresholding with the threshold calculated by the Otsu method is executed with the same `cv2.threshold(...)` function. To use the Otsu threshold method you have to use the `THRESH_OTSU` parameter as a thresholding method. This function can be used to calculate the Otsu threshold without thresholding an image as well, for this we have to run it in a dry run mode:

```
tr, ret = cv2.threshold(I, 0, 1, cv2.THRESH_OTSU +
cv2.THRESH_DRYRUN)
print("The Otsu threshold value is {}".format(tr))
```

Let's try executing the thresholding with the threshold calculated by the Otsu method.

```
In [6]: # Otsu thresholding
tr, I1otsu = cv.threshold(I1gray, 0, 1, cv.THRESH_OTSU)
ShowImages([("Otsu thresholding", I1otsu)])
```

Otsu thresholding



1.2.4 Adaptive thresholding

Adaptive methods do not work with the entire image, but only with its fragments. Such approaches are often used when working with images that represent non-uniformly illuminated objects.

In OpenCV adaptive thresholding is performed with the

`cv2.adaptiveThreshold(image, maxval, algorithm, method, blockSize, c)` function. It supports two adaptive thresholding algorithms:

- `ADAPTIVE_THRESH_MEAN_C` uses simple rectangular kernel mean;
- `ADAPTIVE_THRESH_GAUSSIAN_C` uses a kernel with Gauss weights.

These algorithms support two thresholding methods:

- `cv2.THRESH_BINARY` to set everything greater than the threshold value to the and everything else to ;
- `cv2.THRESH_BINARY_INV` to set everything greater than the threshold value to the and everything else to .

Kernel size is defined by the `blockSize` parameter, and the parameter value `c` is subtracted from the mean value calculated within a window.

```
In [7]: I1ta = cv.adaptiveThreshold(I1gray, 255, cv.ADAPTIVE_THRESH_GAUSSIAN_C, cv.T
ShowImages(["Adaptive thresholding", I1ta])
```

Adaptive thresholding



1.2.5 Self-work

Self-work

Take some arbitrary image and threshold it with **four listed** automatic threshold calculation method:

- Mean value threshold

- Intensity gradient weighted threshold
- Otsu threshold
- Adaptive threshold

Display the resulting images.

Notes:

1. To calculate the mean value fast you should use existing NumPy functions.
2. To make gradient calculation faster you may consider using the `cv2.copyMakeBorder(image, 1, 1, 1, 1, cv2.BORDER_REPLICATE)` function to create a copy of an image with 1 pixel border that replicates the border pixel value to extension region. Then you can use this image with a 1-pixel border to calculate gradients fast with NumPy functions.

```
In [8]: gray = cv.imread("images/coins.jpg", cv.IMREAD_GRAYSCALE)

def mean_threshold(img):
    return (int(img.max()) + int(img.min())) / 2.0

def gradient_threshold(img):
    ext = cv.copyMakeBorder(img, 1, 1, 1, 1, cv.BORDER_REPLICATE).astype(np.float32)
    horiz = np.abs(ext[1:-1, 2:] - ext[1:-1, :-2])
    vert = np.abs(ext[2:, 1:-1] - ext[:-2, 1:-1])
    grad = np.maximum(horiz, vert)
    return float((img * grad).sum() / grad.sum())

t1 = mean_threshold(gray)
bin_mean = (gray > t1).astype(np.uint8) * 255

t2 = gradient_threshold(gray)
bin_grad = (gray > t2).astype(np.uint8) * 255

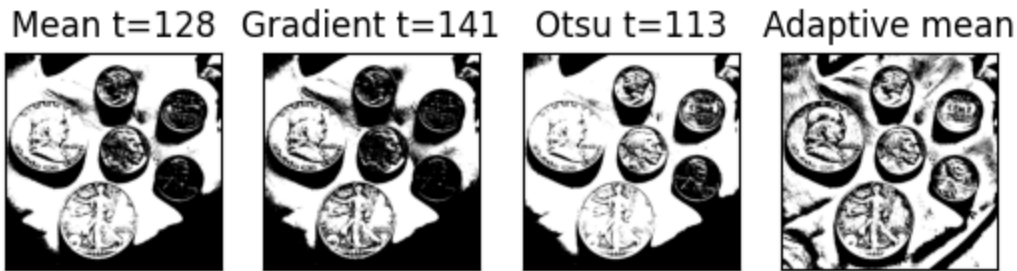
t3, bin_otsu = cv.threshold(gray, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)

bin_adapt = cv.adaptiveThreshold(gray, 255, cv.ADAPTIVE_THRESH_MEAN_C, cv.THRESH_BINARY, 11, 3)

print(f"mean={t1:.1f}, gradient={t2:.1f}, otsu={t3:.1f}")

ShowImages([(f"Mean t={t1:.0f}", bin_mean),
             (f"Gradient t={t2:.0f}", bin_grad),
             (f"Otsu t={t3:.0f}", bin_otsu),
             ("Adaptive mean", bin_adapt)], 4)

mean=127.5, gradient=140.7, otsu=113.0
```



Task 2. Color Segmentation

Take an arbitrary image containing the face(s). Perform the image segmentation according to the Weber principle. Perform the image segmentation based on the skin color and try different formulas on photos with various photo illumination conditions.

Let's consider several basic methods of image segmentation.

2.1 Weber principle

The *Weber principle* tells us that there is a not noticeable distance for the human eye and depending on the intensity value, this range can be calculated using the formula:

$$\frac{I - I_0}{I_0} \leq \frac{1}{2}$$

So, the *Weber principle* assumes that the human eye does not perceive well the difference in gray levels between I_0 and $I_0 + \frac{I_0}{2}$, where $f(I)$ is the Weber function, k is the class number, $F(I)$ is the piecewise non-linear grayscale function. Based on this principle the segmentation algorithm is designed for the segmentation of grayscale images.

The Webber principle segmentation algorithm consists of the following steps:

1. Initialize initial conditions: first class number $k=1$, all image pixels are not segmented.
2. Find the minimum value of the not-segmented image part and store it to I_0 .
3. Calculate the I_1 value according to the Weber formula and assign the segment k and the value I_0 to all pixels whose intensities are in the range $[I_0, I_1]$.
4. If there are any not-segmented pixels in the image, then increment k ($k = k + 1$) and go to step 2.

5. If there are no not segmented pixels then the segmentation is finished.

The segmentation data can be displayed with artificial colors and a JET color scheme:

```
Ijet = cv.applyColorMap(((I - I.min()).astype(np.float32) * 255 /
                        (I.max() - I.min()).astype(np.uint8)),
cv.COLORMAP_JET)
```

2.1.1 Self-work

Self-work

Implement the image segmentation algorithm based on the Weber principle. Display the resulting images.

```
In [9]: def weber_width(level):
        level = float(level)
        if level <= 88:
            return 20.0 - 12.0 * level / 88.0
        if level <= 138:
            return 0.002 * (level - 88) ** 2
        return 7.0 * (level - 138) / 117.0 + 13.0

src_weber = cv.imread("images/coins.jpg", cv.IMREAD_GRAYSCALE)

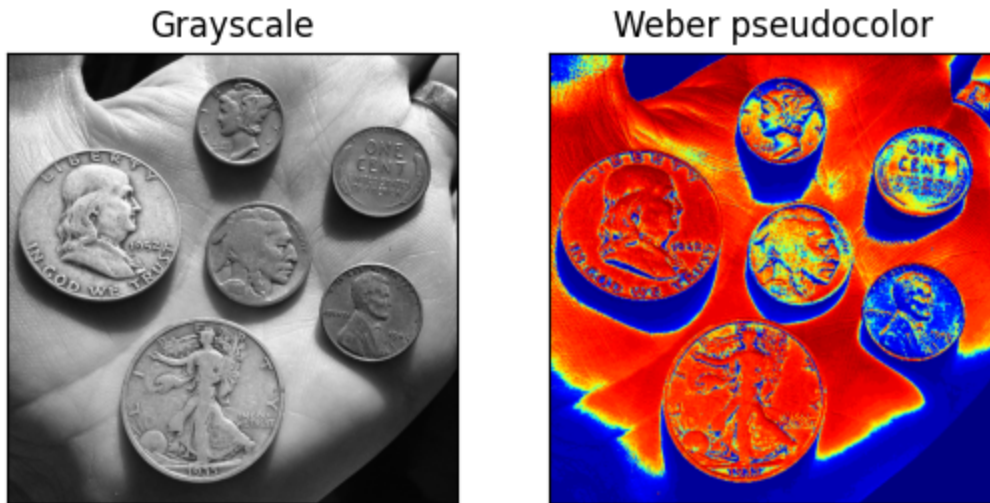
seg_map = np.full(src_weber.shape, -1, dtype=np.int32)
cls = 0
while np.any(seg_map < 0):
    remaining = src_weber[seg_map < 0]
    base = int(remaining.min())
    width = weber_width(base)
    upper = base + width
    region = (seg_map < 0) & (src_weber >= base) & (src_weber <= upper)
    seg_map[region] = cls
    cls += 1

print(f"Total Weber classes: {cls}")

norm = ((seg_map - seg_map.min()) * 255.0 / (seg_map.max() - seg_map.min()))
pseudo = cv.applyColorMap(norm, cv.COLORMAP_JET)

ShowImages([("Grayscale", src_weber), ("Weber pseudocolor", pseudo)], 2)
```

Total Weber classes: 39



2.2 Skin Color Segmentation

The general principle of skin color segmentation is determining the criterion for the proximity of the pixel intensity to the skin tone. It is quite difficult to describe the *skin tone* analytically since its description is based on the human perception of color. Moreover, it changes with lighting, differs among different nationalities, etc.

There are several analytical descriptions for images in the RGB color space that allow a pixel to be assigned to the *skin* class of the following conditions. For example, at uniform daylight illumination, the following conditions may be used to segment a part of an image as skin:

In the case of the flashlight or daylight lateral illumination:

Also, there is a formula that is based on normalized RGB values (which are normalized by the sum of):

$$\frac{\frac{R}{R+G+B}}{\frac{\frac{G}{R+G+B}}{B}} = \frac{r}{g}$$

$$\frac{r}{g} = \frac{rb}{(r+g+b)^2}$$

$$\frac{rg}{(r+g+b)^2}$$

To implement these segmentation methods with OpenCV we should use NumPy matrix operations and create a mask to segment skin. Let us implement the skin color segmentation for the first of these three formulas.

```
In [10]: # We use an offline skimage sample image with a face instead of a network re
I2 = cv.imread("images/portrait.jpg", cv.IMREAD_COLOR)

# Split into layers
I2BGR = cv.split(I2)

# Build up the mask
I2skin = np.logical_and.reduce(
    [I2BGR[2] > 95, # R > 95
     I2BGR[1] > 40, # G > 40
     I2BGR[0] > 20, # B > 20
     np.maximum.reduce(I2BGR) - np.minimum.reduce(I2BGR) > 15, # Max(|R-G|) > 15
     np.abs(I2BGR[2] - I2BGR[1]) > 15, # |R - G| > 15
     I2BGR[2] > I2BGR[0], # R > B
     I2BGR[2] > I2BGR[1]] # G > B

# Create a copy of the source image and remove everything except for the face
I2copy = I2.copy()
I2copy[~I2skin] = 0

# Display the result
ShowImages(["Source image", I2),
           ("Skin segment", I2skin),
           ("Skin only", I2copy)], 3)
```

Source image



Skin segment



Skin only



2.2.1 Self-work

Self-work

Implement the **two** remaining skin segmentation algorithms. Display the resulting images.

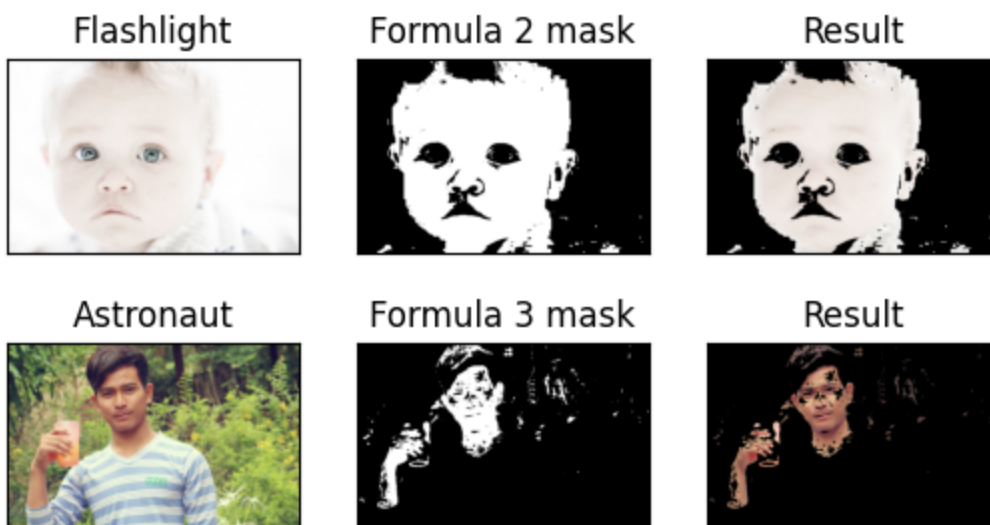
Notes.

1. Formula 2 works well only for direct flashlight illumination, so it would segment nothing on most of the regular images. You may use "images/flashlight.jpg" image to check your result.

```
In [11]: def apply_mask(img, mask):
    out = img.copy()
    out[~mask] = 0
    return out

img_flash = cv.imread("images/flashlight.jpg", cv.IMREAD_COLOR)
b2, g2, r2 = [c.astype(np.int16) for c in cv.split(img_flash)]
mask_flash = (r2 > 220) & (g2 > 210) & (b2 > 170) & (np.abs(r2 - g2) <= 15)
ShowImages(["Flashlight", img_flash,
            ("Formula 2 mask", mask_flash.astype(np.uint8)),
            ("Result", apply_mask(img_flash, mask_flash))], 3)

img_norm = cv.imread("images/portrait.jpg", cv.IMREAD_COLOR)
bf, gf, rf = [c.astype(np.float64) for c in cv.split(img_norm)]
total = rf + gf + bf + 1e-6
rn, gn, bn = rf / total, gf / total, bf / total
denom = np.power(rn + gn + bn, 2)
mask_norm = (rn / (gn + 1e-6) > 1.185) & (rn * bn / denom > 0.107) & (rn * g
ShowImages(["Astronaut", img_norm,
            ("Formula 3 mask", mask_norm.astype(np.uint8)),
            ("Result", apply_mask(img_norm, mask_norm))], 3)
```



Task 3. Chromatic Segmentation

Take an arbitrary image containing a limited number of colored objects. Perform image segmentation in the uniform color space by the nearest neighbors method. Perform image segmentation in the CIE Lab color space by the k -means method.

In the CIE Lab color space, the value of lightness is separated from the value of the chromatic components of the color (hue, saturation). The lightness of a color is given by the L^* coordinate, which can range from 0 (dark) to 100 (light). The chromatic component of a color is given by two Cartesian coordinates a^* (which defines the color position in the range from green to red) and b^* (which defines the color position in the range from blue to yellow). In CIE Lab color space the grayscale colored pixels of an image have zero a^* and b^* coordinates. We can use the CIE Lab color space to perform the segmentation of an image based only on chromatic color components and ignore lightness.

3.1 Segmentation in CIE Lab color space

The algorithm's general idea is to divide a color image into segments of dominant colors. For this, we will let the user select a set of dominant colors, then search for a distance to each of these colors and segment image pixels by the shortest distance.

At first, we have to let the user select points on the image. To do it we will use the OpenCV GUI library and create a window with the mouse callback function `MouseHandler()`. This function will register each left mouse button click and add corresponding pixel coordinates to an array with selected points. This will continue until the window is shown or the user presses an ESC button.

So, the general workflow is as follows:

1. Create a GUI window with `cv2.imshow(window_name, I)` function.
2. Register a mouse callback with `cv2.setMouseCallback(window_name, MouseHandler, params)`.
3. Run the infinite loop and break it in case if:
 - Window is closed (checked by `cv2.getWindowProperty(window_name, cv2.WND_PROP_VISIBLE) < 1`);
 - Or the ESC button is pressed (checked by `cv.waitKey(100) == 27`).

In the mouse handler (`MouseHandler()`) function we check the event type, append click coordinates to the selected points list, and update the shown image by making a new image and calling `cv2.imshow(window_name, I)` function again.

```
In [12]: I3 = cv.imread("images/objects.jpg", cv.IMREAD_COLOR)

radius = 12

# Predefined reference points on differently colored regions (no interactive
```

```

points = [(126, 331),
          (231, 224),
          (434, 294)]

I3copy = I3.copy()
for p in points:
    cv.circle(I3copy, p, radius, (0, 0, 255), 2)
ShowImages(["Labels", I3copy])

```

Labels



Now that we have the list of the selected points we may proceed with the general segmentation algorithm:

1. Convert an image to CIE Lab color space.
2. Take an area around selected pixels and calculate the mean value in this area to get the reference color for each segment.
3. Calculate the Euclidean distance in layers ($\sqrt{\frac{L^2}{2} + \frac{a^2}{2} + \frac{b^2}{2}}$) for each image pixel from the selected points.
4. For each image pixel select the nearest color in coordinate space. This will be the desired segmentation.

```

In [13]: # 1. Convert to CIE Lab
I3lab = cv.cvtColor(I3, cv.COLOR_BGR2LAB)
# Split it into layers
I3lab = cv.split(I3lab)

```

```

In [14]: # 2. Instead of taking the selected pixel value
# We will calculate the mean color value in an area around the pixel
points_ab = []

```

```

points_bgr = []
for p in points:
    mask = np.zeros_like(I3lab[0])
    cv.circle(mask, p, radius, 255, -1)
    a = I3lab[1].mean(where = mask > 0)
    b = I3lab[2].mean(where = mask > 0)
    points_ab.append((a, b))
    points_bgr.append(I3[mask > 0, :].mean(axis=(0)))

```

```

In [15]: # 3. Calculate distance
distances = []
for ab in points_ab:
    distances.append(np.sqrt(
        np.power(I3lab[1] - ab[0], 2) +
        np.power(I3lab[2] - ab[1], 2)))

# Calculate the minimum distance among them all
distance_min = np.minimum.reduce(distances)

```

```

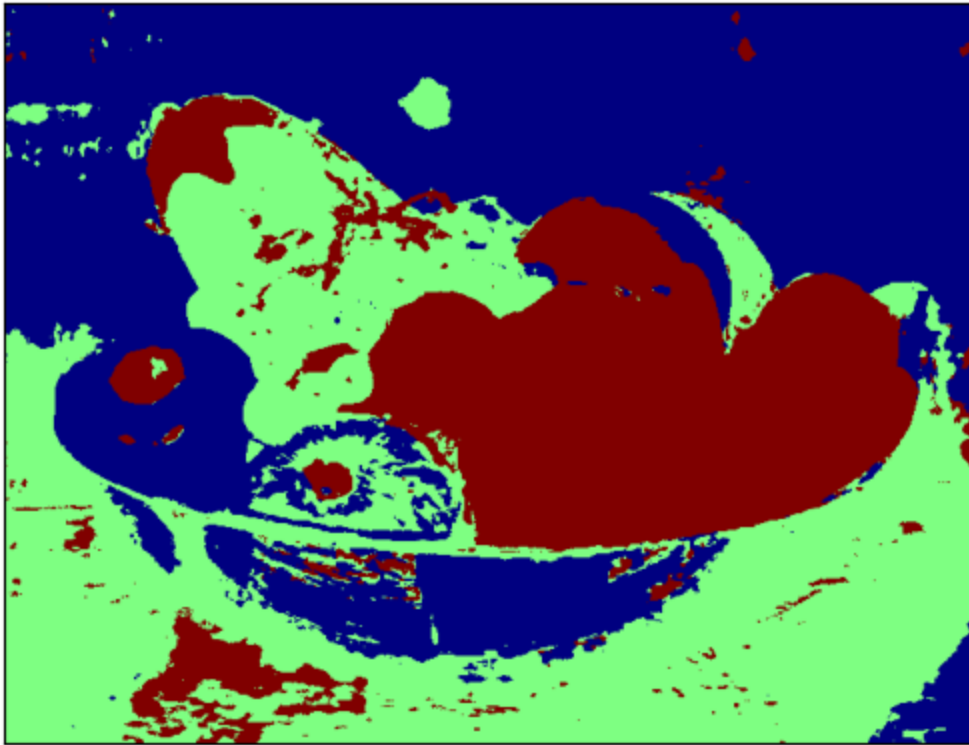
In [16]: # 4. Create segmented areas
I3labels = np.zeros_like(I3lab[0], dtype = np.uint8)

I3segments = []
# Fill them for each selected color
for i in range(len(points_ab)):
    Itmp = np.zeros_like(I3)
    # Segmentation mask is created by finding all pixels with a matching distance
    mask = distance_min == distances[i]
    # Add labels to the segmentation plot
    I3labels[mask] = i
    # Segmented area of source image
    Itmp[mask] = I3[mask]
    # Add the segmented area to a list of segments
    I3segments.append("Segment {}".format(i), Itmp)

# Display it
ShowImages(["Labels", cv.applyColorMap(((I3labels - I3labels.min()).astype(
    (I3labels.max() - I3labels.min()))).
ShowImages(I3segments, 2)

```

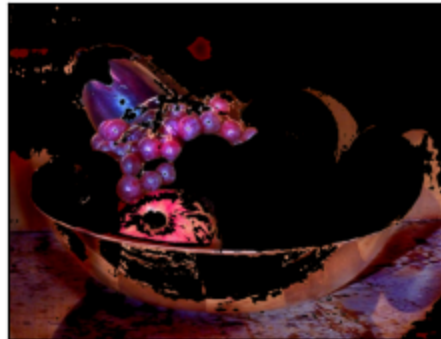
Labels



Segment 0



Segment 1



Segment 2

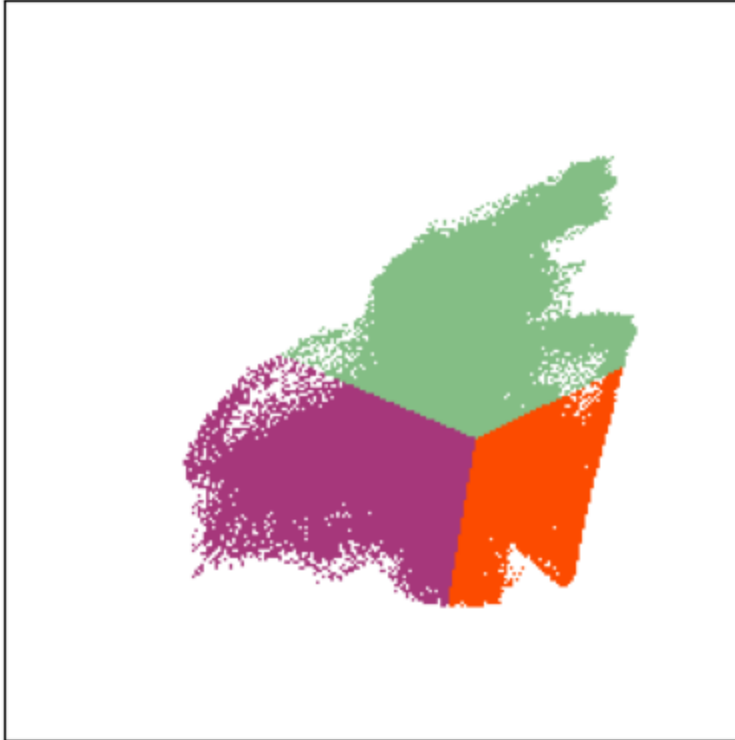


Finally, we can place the distribution of image pixel colors to a plot in the coordinate system. The plot pixel color is defined by a mean μ which was calculated along with calculating the mean μ value and stored in the `points_bgr` array.

```
In [17]: # The color distribution plot
I3plot = np.full((256, 256, 3), 255, dtype = np.uint8)
for i in range(len(points_ab)):
    mask = I3labels == i
    I3plot[I3lab[1][mask], I3lab[2][mask], :] = points_bgr[i]

# Display it
ShowImages(["Color plot", I3plot])
```

Color plot



3.1.1 Self-work

Self-work

Try modifying the above algorithm to use the HSV color model.

Notes.

1. To convert from BGR to HSV color space you should use the `cv2.COLOR_RGB2HSV` color model conversion parameter.

```
In [18]: hsv = cv.cvtColor(I3, cv.COLOR_BGR2HSV)
H = hsv[:, :, 0].astype(np.float32)
S = hsv[:, :, 1].astype(np.float32)

refs = []
for (px, py) in points:
    spot = np.zeros(H.shape, dtype=np.uint8)
    cv.circle(spot, (px, py), radius, 255, -1)
```

```

refs.append((H[spot > 0].mean(), S[spot > 0].mean()))

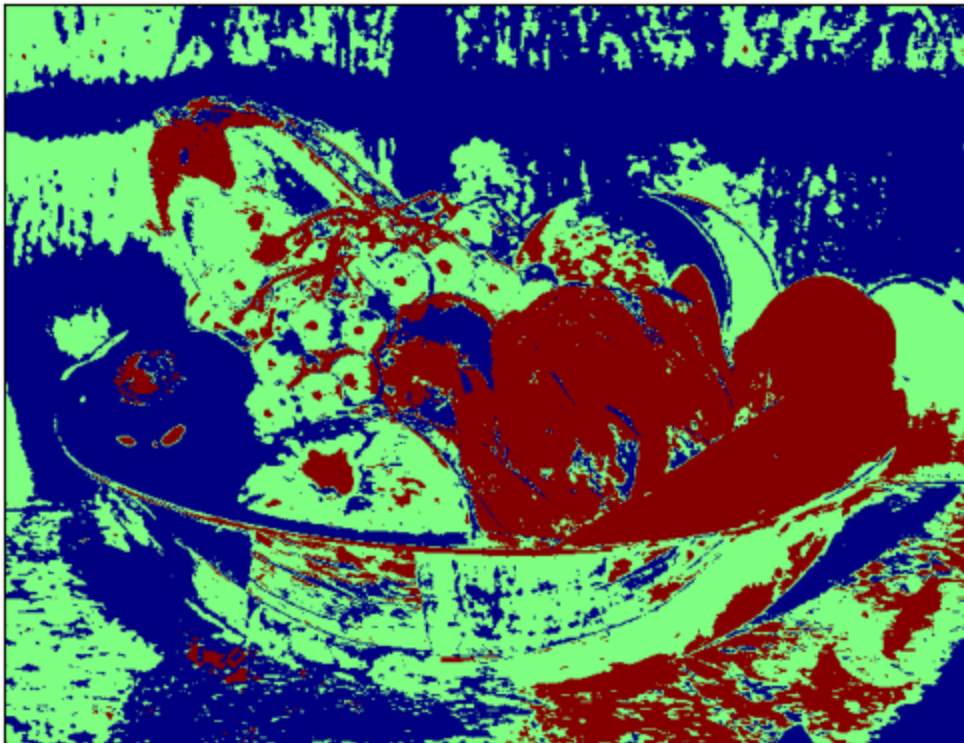
dist_stack = []
for (hr, sr) in refs:
    dh = np.abs(H - hr)
    dh = np.minimum(dh, 180.0 - dh)
    dist_stack.append(dh * dh + (S - sr) ** 2)
dist_stack = np.stack(dist_stack, axis=0)
label_map = np.argmin(dist_stack, axis=0).astype(np.uint8)

seg_list = []
for idx in range(len(refs)):
    piece = np.zeros_like(I3)
    sel = label_map == idx
    piece[sel] = I3[sel]
    seg_list.append((f"Segment {idx}", piece))

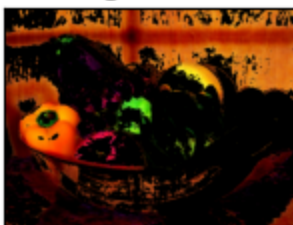
color_labels = cv.applyColorMap((label_map * (255 // max(1, len(refs) - 1))))
ShowImages(["HSV labels", color_labels])
ShowImages(seg_list, 3)

```

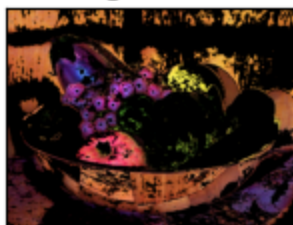
HSV labels



Segment 0



Segment 1



Segment 2



3.2 k-means Clustering

The idea of the method is to determine the centers of k -clusters and assign to each cluster the pixels closest to these centers. All pixels are considered as vectors $\vec{x}_i$.

The segmentation algorithm consists of the following steps:

1. Randomly determine vectors $\vec{c}_j$, which are declared as initial centers of clusters.
2. Update mean values of the vectors $\vec{c}_j$ by calculating distances from each vector $\vec{x}_i$ to each $\vec{c}_j$ and their classification according to the criterion of minimal distance from the vector to the cluster, recalculation of average values $\vec{c}_j$ across all clusters.
3. Repeat step until the cluster centers stop changing.

The implementation of the method is very similar to the previous approach and contains a number of similar actions. We will work in the CIE Lab color space, so the first step is transformation from the BGR space to the Lab and splitting into layers:

```
Ilab = cv2.cvtColor(I, cv2.COLOR_BGR2LAB)
Ilab = cv2.split(Ilab)
```

Then we have to merge `a` and `b` layers of the Lab representation of our source image and use the NumPy reshaping function to create the two-dimensional array of the image pixel colors:

```
ab = cv2.merge([Ilab[1], Ilab[2]])
ab = ab.reshape(-1, 2).astype(np.float32)
```

This two-dimensional array can be then passed to the `cv2.kmeans()` function that performs the k -means clustering. Parameters allow us to define stop criteria and the number of attempts to select a set of starting points. In the following code stop criteria is defined as not more than 10 iterations of difference between steps less than 1. Starting points are selected randomly (due to the `cv2.KMEANS_RANDOM_CENTERS` flag being used) and selection is done 10 times. After algorithm execution is finished, the returned `labels` parameter is reshaped back to an original image shape:

```
k = 3
criteria = (cv2.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 10, 1.0)
ret, labels, centers = cv2.kmeans(ab, k, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)
labels = labels.reshape((Ilab[0].shape))
```

Then, it is possible to use the generated labels to segment the image into a set of images or masks:

```
segmentedFrames = []
for i in range(k):
    Itmp = np.zeros_like(I)
    mask = labels == i
```

```
Itmp[mask] = I[mask, :]
segmentedFrames.append(Itmp)
```

3.2.1 Self-work

Self-work

Implement the k-means clustering algorithm and display the result.

```
In [19]: img_km = cv.imread("images/scene_color.jpg", cv.IMREAD_COLOR)
lab = cv.cvtColor(img_km, cv.COLOR_BGR2LAB)
a_ch, b_ch = lab[:, :, 1], lab[:, :, 2]

features = np.stack([a_ch, b_ch], axis=-1).reshape(-1, 2).astype(np.float32)

n_clusters = 4
stop = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 20, 0.5)
compactness, lbl, ctr = cv.kmeans(features, n_clusters, None, stop, 15, cv.K_
lbl = lbl.reshape(a_ch.shape)

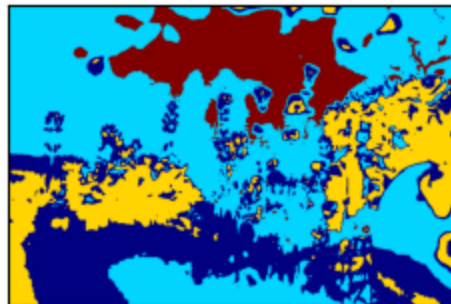
clusters = []
for c in range(n_clusters):
    layer = np.zeros_like(img_km)
    m = lbl == c
    layer[m] = img_km[m]
    clusters.append((f"Cluster {c}", layer))

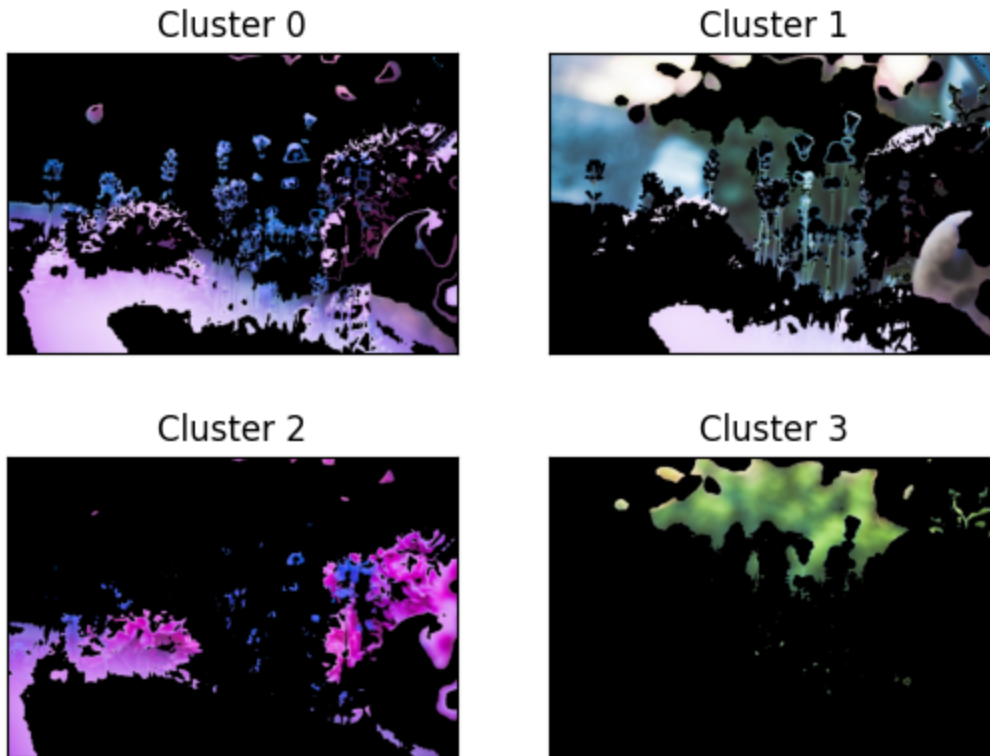
ShowImages([("Source (coffee)", img_km),
            ("k-means labels", cv.applyColorMap((lbl * (255 // (n_clusters -
ShowImages(clusters, 2)
```

Source (coffee)



k-means labels





Task 4. Texture Segmentation

Take an arbitrary image containing two heterogeneous textures. Perform texture segmentation of the image.

In texture segmentation, three main approaches are used to describe texture: statistical, structural, and spectral. In the practical assignment, we will consider a statistical approach that describes the segment texture as smooth, rough, or grainy.

We will consider the image intensity as a random variable , which corresponds to the distribution probability calculated from the image histogram. The *Central moment* of order of a random variable is the parameter μ_n calculated by the formula:

$$\mu_n = \frac{1}{N} \sum_{i=0}^{L-1} (i - \mu_1)^n p_i$$

where

- L is the number of intensity levels;
- μ_1 is the mean value of a random variable :

$$\mu_1 = \frac{1}{N} \sum_{i=0}^{L-1} i p_i$$

The expression above implies that σ_0 and σ_1 . To describe the texture, the *variance* of a random variable is important, which is equal to the second moment σ^2 and is a measure of the brightness contrast. It can be used to calculate the features of *smoothness*.

So, let us introduce a measure of relative smoothness :

$$\frac{\sigma^2}{\sigma_{\max}^2}$$

The relative smoothness is zero for areas with constant intensity (zero variance) and approaches unity for large variances σ^2 . For grayscale images with an intensity range $[0, 255]$, it is necessary to normalize the variance to the range $[0, 1]$, since the values of the variances will be too large for the initial range. Normalization is carried out by dividing the variance σ^2 by 255^2 .

The *standard deviation* is also often used as a texture characteristic:

The third moment is the *histogram symmetry characteristic*. To estimate texture features, the *entropy* function is used, which determines the spread of neighboring pixels intensities:

$$\sum_{i=0}^{L-1} p_i^2$$

Another important characteristic that describes the texture is the *uniformity measure*, which evaluates the uniformity of the histogram:

$$\sum_{i=0}^{L-1} \frac{1}{2^i}$$

Let us try doing segmentation using *Entropy* of a pixel neighborhood as a texture characteristic. For this we have to calculate the image Entropy first, then threshold it to get the starting morphology. Unfortunately, OpenCV does not provide the corresponding function for Entropy calculation, however, in the case of Python it can be found in the scikit-image library named `skimage.filters.rank.entropy()`. To define the neighboring area we will use the rectangular kernel created by `skimage.morphology.square()` function. Since scikit-image converts an image to `float64` type we have to do a backward conversion along with normalization to a range.

```
In [20]: # Read an image from file
fn = "images/texture.jpg"
I4 = cv.imread(fn, cv.IMREAD_COLOR)
```

```

if not isinstance(I4, np.ndarray) or I4.data == None:
    print("Error reading file \"{}\\".format(fn))
    exit()

# Convert to grayscale
I4gray = cv.cvtColor(I4, cv.COLOR_BGR2GRAY)
# And display
ShowImages([("Source", I4), ("Grayscale", I4gray)], 2)

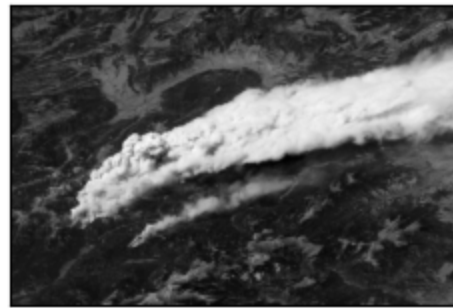
# Calculate entropy
I4e = skimage.filters.rank.entropy(I4gray, skimage.morphology.square(9)).ast
# Normalize it to [0, 1] range
I4en = (I4e - I4e.min()) / (I4e.max() - I4e.min())
# And display
ShowImages([("Entropy", I4en)], 2)

```

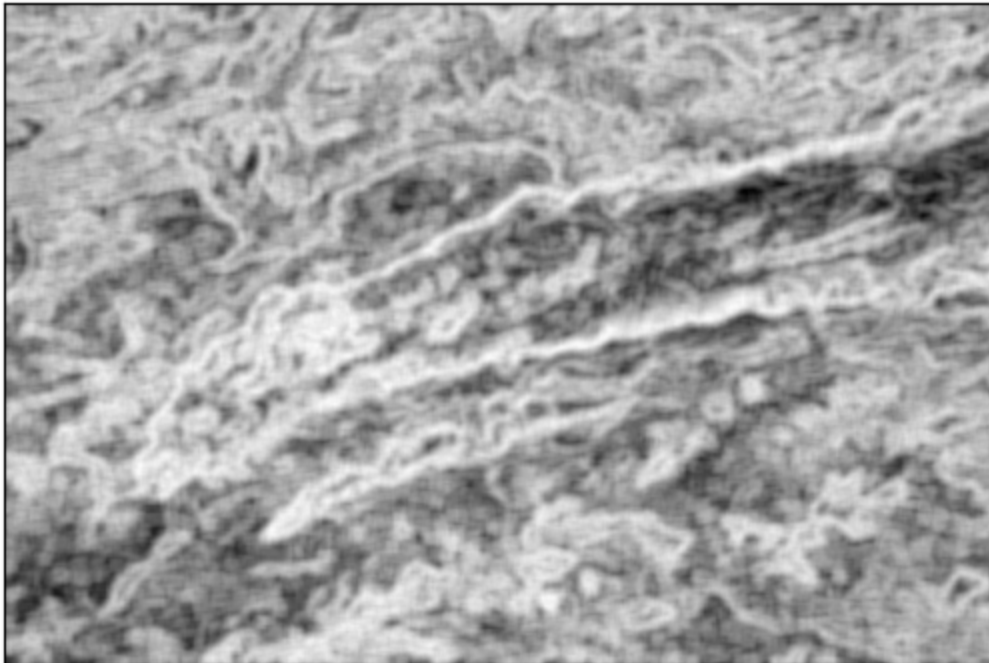
Source



Grayscale



Entropy



Next, we have to threshold the entropy image and filter the resulting morphology. For thresholding, we will use the Otsu thresholding, while the morphological filtering will be performed in three steps:

1. At first, remove connected regions (equivalent to MATLAB's `bwareaopen()` function);
2. Secondly, remove internal defects with closing operation (executed by `cv2.morphologyEx()` function with `cv2.MORPH_CLOSE` parameter) and rectangular structure element of size (created by `cv2.getStructuringElement()` function with shape parameter `cv2.MORPH_RECT`);
3. And, thirdly, fill remaining *holes* (equivalent to MATLAB's `imfill('holes')` function).

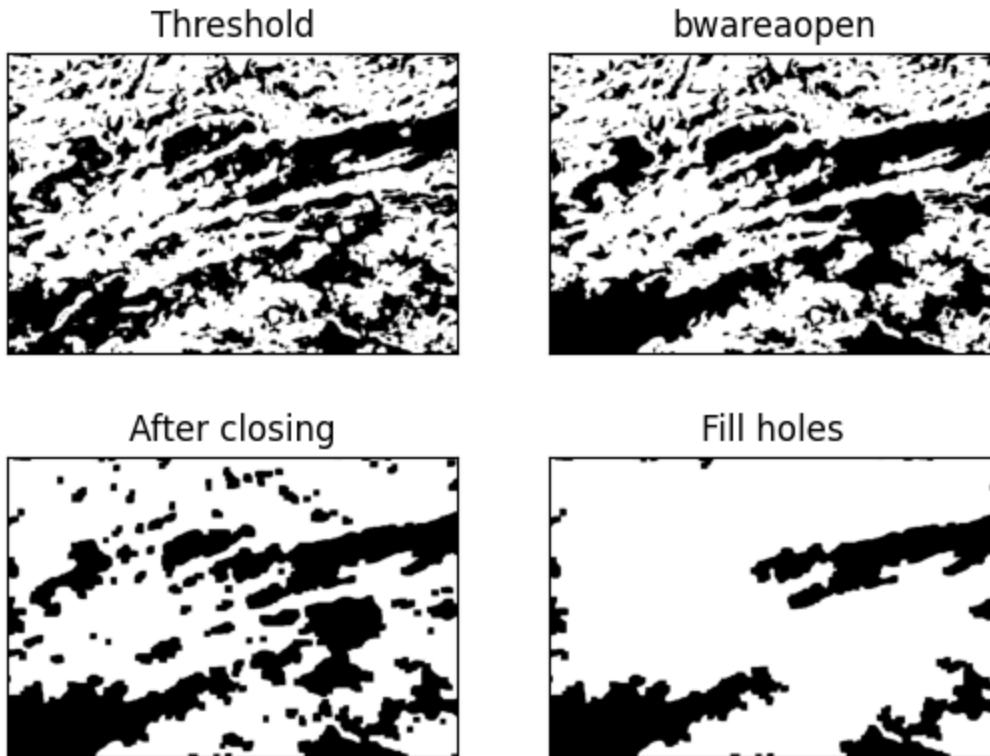
Even if OpenCV lacks MATLAB's `bwareaopen(A, dim)` and `imfill(I, 'holes')` functions, they can be easily implemented using OpenCV's `connectedComponentsWithStats()`. You may check the `pa_utils.py` file for the implementation of these functions with OpenCV. However, now we will simply import and use them.

```
In [21]: from pa_utils import bwareaopen, imfillholes

# Threshold
ret, I4t = cv.threshold(np.uint8(I4en * 255), 0, 255, cv.THRESH_OTSU)

# Remove connected regions
I4bwao = bwareaopen(I4t, 2000)
# Remove internal defects with closing
nhood = cv.getStructuringElement(cv.MORPH_RECT, (9, 9))
I4close = cv.morphologyEx(I4bwao, cv.MORPH_CLOSE, nhood)
# Fill holes to get the final mask
I4mask = imfillholes(I4close)

ShowImages([("Threshold", I4t), ("bwareaopen", I4bwao), ("After closing", I4
```

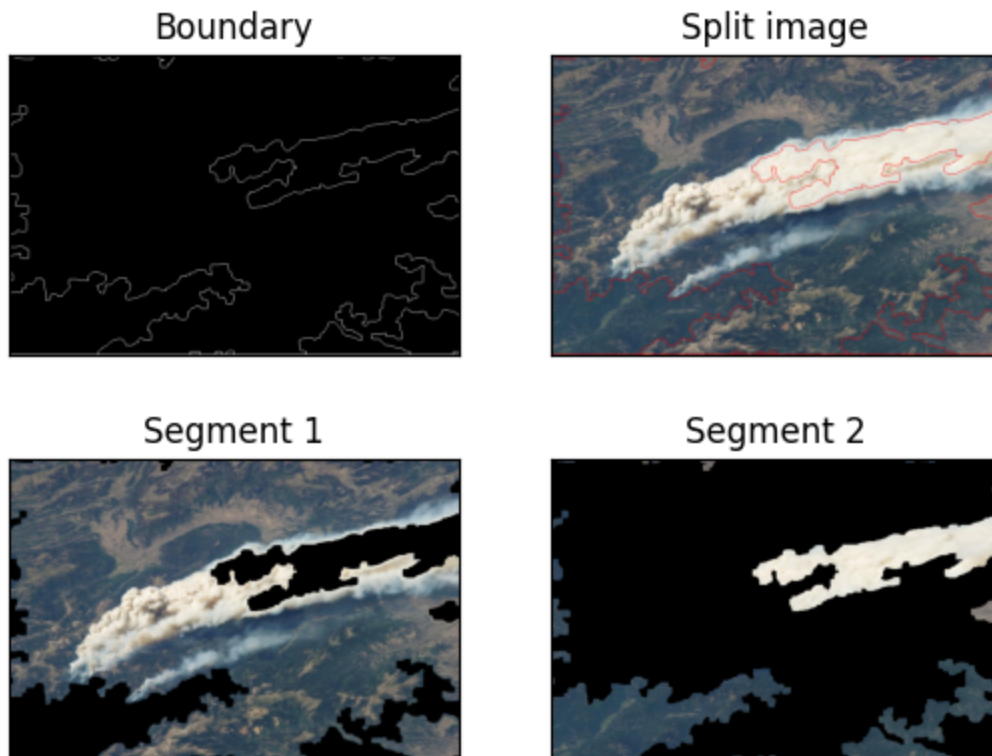


Next, using the OpenCV `cv2.findContours()` function it's possible to find shape contours and then draw them on a black background with the `cv2.drawContours()` function to define the contour mask. Then we can use this mask to apply the border to the source image.

```
In [22]: # Find boundary
contours, h = cv.findContours(I4mask, cv.RETR_TREE, cv.CHAIN_APPROX_NONE)
I4boundary = np.zeros_like(I4mask)
cv.drawContours(I4boundary, contours, -1, 255, 1)
# Outline the whole image as well
cv.rectangle(I4boundary, (0, 0), (I4.shape[1] - 1, I4.shape[0] - 1), 255)

# Separate segments
I4seg1 = I4.copy()
I4seg1[I4mask == 0] = 0
I4seg2 = I4.copy()
I4seg2[I4mask != 0] = 0
I4split = I4.copy()
I4split[I4boundary != 0] = [0, 0, 255]

ShowImages([("Boundary", I4boundary),
             ("Split image", I4split),
             ("Segment 1", I4seg1),
             ("Segment 2", I4seg2)], 2)
```



4.1 Self-work

Self-work

Take an arbitrary image and implement the image texture segmentation.
Display the results.

Optional (1 extra point)

Use another statistical texture parameter instead of Entropy when implementing the texture segmentation.

```
In [23]: from pa_utils import bwareaopen, imfillholes

def to_gray(arr):
    g = arr if arr.ndim == 2 else cv.cvtColor(arr, cv.COLOR_RGB2GRAY)
    return cv.resize(g, (256, 256))

left = to_gray(cv.imread("images/texture_a.jpg", cv.IMREAD_COLOR))
right = to_gray(cv.imread("images/texture_b.jpg", cv.IMREAD_COLOR))
composite = np.hstack([left, right])

win = 15
mean_local = cv.boxFilter(composite.astype(np.float32), -1, (win, win))
mean_sq = cv.boxFilter(composite.astype(np.float32) ** 2, -1, (win, win))
local_std = np.sqrt(np.maximum(mean_sq - mean_local ** 2, 0))
std_norm = (local_std - local_std.min()) / (local_std.max() - local_std.min())

ShowImages([("Composite texture", composite), ("Local std (sigma)", std_norm
```

```

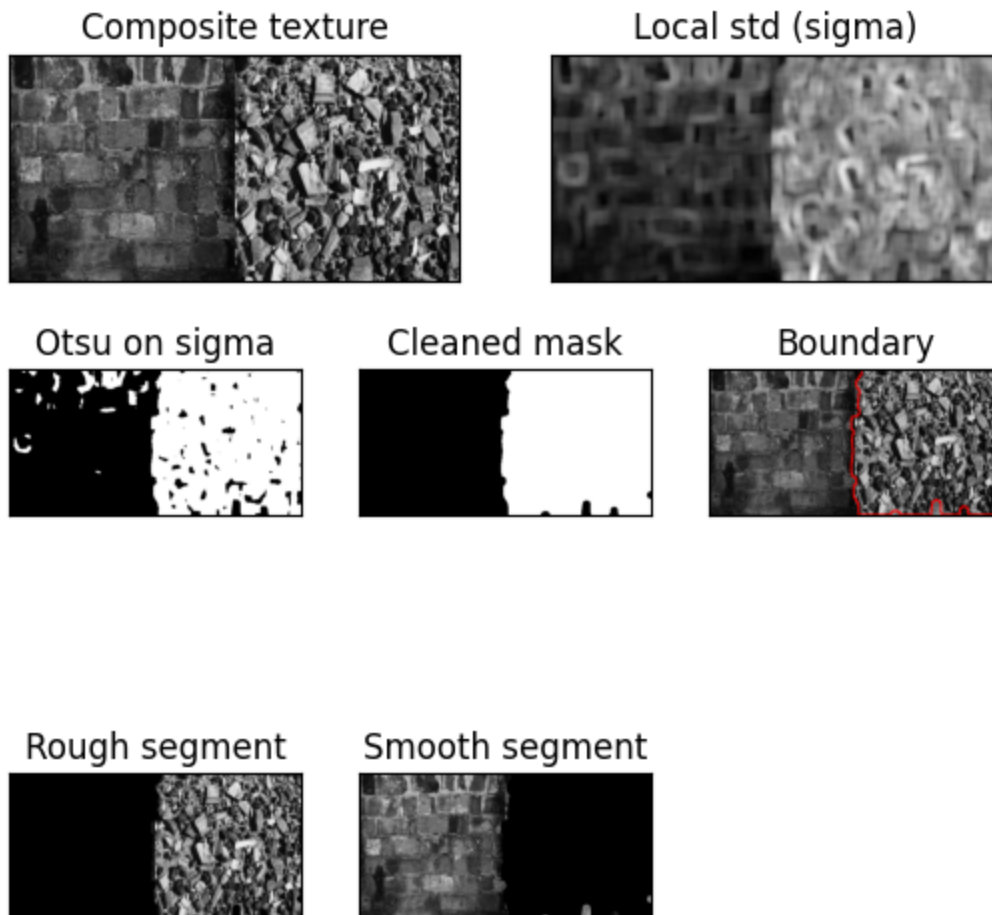
ret, raw = cv.threshold(np.uint8(std_norm * 255), 0, 255, cv.THRESH_OTSU)
cleaned = bwareaopen(raw, 1500)
se = cv.getStructuringElement(cv.MORPH_ELLIPSE, (13, 13))
closed = cv.morphologyEx(cleaned, cv.MORPH_CLOSE, se)
final_mask = imfillholes(closed)

contours, _ = cv.findContours(final_mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_
overlay = cv.cvtColor(composite, cv.COLOR_GRAY2BGR)
cv.drawContours(overlay, contours, -1, (0, 0, 255), 2)

seg_a = composite.copy()
seg_a[final_mask == 0] = 0
seg_b = composite.copy()
seg_b[final_mask != 0] = 0

ShowImages([("Otsu on sigma", raw), ("Cleaned mask", final_mask),
            ("Boundary", overlay), ("Rough segment", seg_a), ("Smooth segmer

```



Questions

Please answer the following questions:

- When is it appropriate to use Weber segmentation?

Подходит для grayscale и для случая, когда яркость надо разложить на уровни по восприятию, а не равномерно. По Веберу заметность перепада зависит от фона: темнее фон — тоньше класс. Отсюда применение — перцептивное квантование, карты уровней, предобработка. Если объекты различаются цветом или фактурой, метод не сработает.

- *What are the a and b color coordinates values for a grayscale image in the CIE Lab color space?*

Ноль. Серому соответствуют $a = 0$ и $b = 0$, всё значение сидит в светлоте L . В OpenCV из-за 8-битной упаковки этот ноль лежит в 128.

- *What is the reason for performing an image segmentation in the CIE Lab color space instead of the original RGB one?*

RGB смешивает яркость и цвет: при смене света R , G и B меняются вместе, и пороги по цвету плывут. Lab разводит их — L отвечает за свет, а a и b за цветность, так что блики и тени почти не мешают. Вдобавок расстояние в плоскости a, b лучше согласуется с человеческим ощущением цветовой разницы.

Conclusion

What have you learned with this task? Don't forget to conclude it.

Работала на разных фотографиях — городская сцена, монеты, портреты, фрукты, цветы и две текстуры. Так понятнее, что метод привязан не к конкретному файлу, а к свойствам сцены.

Бинаризацию проверила во всех вариантах: одиночный порог, двойной и четыре автоматических. На монетах сразу видно слабое место глобального порога — неравномерный фон он режет грубо, адаптивный аккуратнее. По цвету собрала веберовскую сегментацию и обе недостающие формулы кожи (вспышечную гоняла на flashlight.jpg).

Ближайший цвет переписала под HSV через `argmin` по стеку расстояний, `k-means` взяла с $k = 4$. Текстуру в этот раз делила не по энтропии, а по локальному СКО — посчитала через `boxFilter`, и граница между двумя текстурами получилась чище. Вывод простой: всё держится на правильно выбранном пространстве и признаке, пороги вторичны.